

# SpendPartition — implementation status and measurements

Supplement to the SC6109 project proposal · 30 September 2026

HE HUZHENGXIONG (G2509011E) · LIU KEFAN (G2509368B) · RAVINJEET SINGH DAVINDAR SINGH (G2509014G) · ZHOU CONGXIANG (G2509183J)

The proposal is one page, as the project description asks. This document is the long form of its last section: what is built, what was measured, and how to check both. Everything here comes from the repository at [github.com/liukefan821/spendpartition](https://github.com/liukefan821/spendpartition), and every number was read back from a contract call or taken from a transaction receipt.

## 1. What the contract does

A principal fixes a budget  $B_G$  per time window and registers  $N$  delegates. Delegate  $i$  holds a reserved allocation  $r_i$ , and the remainder  $S = B_G - \sum r_i$  is a surplus that any delegate may draw on. The split is set by one parameter,  $\rho = \sum r_i / B_G$ , given as a fraction so that no floating point enters the contract.

A payment of  $a$  by delegate  $i$  takes  $\min(a, r_i - \text{spent}_i)$  from the reservation and the rest from the surplus. If the surplus cannot cover that rest, the whole payment reverts; there is no partial grant. Windows advance by an epoch tag rather than by a reset: each stored value carries the window it belongs to, and a value tagged with an older window reads as zero. Nothing iterates over the delegate set, and no keeper is needed.

The properties we hold the contract to:

|    |                                                                                                      |
|----|------------------------------------------------------------------------------------------------------|
| I1 | in any window, the sum of all delegates' spending is at most $B_G$                                   |
| I2 | a payment that fits inside a delegate's own unspent reservation is never refused by the budget check |
| I3 | surplus consumption never exceeds $S$                                                                |
| I4 | surplus consumption equals the sum over delegates of $\max(0, \text{spent}_i - r_i)$                 |
| I5 | at the first observation in a new window every effective value is zero                               |

Configuration is fixed at deployment: there is no path to change the delegate set,  $B_G$ ,  $\rho$  or the window length. One token per deployment. Lending an unused allocation with the right to recall it is outside the design, and that is a result rather than an omission: once value has left the contract it cannot be preempted.

## 2. What is in the repository

| Path                            | Contents                                                                                                                                                              |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/SpendPartition.sol          | the deployed contract: two packed storage slots, one SLOAD per payment for the payer and one more only when the payment spills into the surplus                       |
| src/SpendPartitionReference.sol | a second implementation of the same rules, written plainly: state keyed by window id, a linear scan for the delegate index, everything recomputed from its definition |
| src/ISpendPartition.sol         | the shared interface the harnesses use; the deployed contract does not inherit it, so its bytecode and its gas numbers stay fixed                                     |
| test/                           | 53 tests across nine suites, plus mocks and four mutants                                                                                                              |
| script/GasSweep.s.sol           | the five-phase gas sweep, broadcast to a local chain                                                                                                                  |
| analysis/                       | the receipt parser, the dashboard template and its builder                                                                                                            |
| results/container_2026-09-30/   | a full reference run: test log, gas and sweep data, both figures, and the toolchain versions                                                                          |
| docs/frozen/                    | the specification, property-test plan and storage layout, frozen on 2026-08-08 before implementation began                                                            |

Toolchain, pinned in foundry.toml: solc 0.8.30, EVM version prague, optimizer on with 200 runs, OpenZeppelin v5.7.0, forge-std v1.16.2, fuzz seed 0x5eed.

## 3. Verification

### 3.1 The suite

| Suite                  | Tests | What it covers                                                                                                                                                                                                                                                                                                                    |
|------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scenarios.t.sol        | 19    | the traces computed by hand in the test plan: apportionment at $B_G = 100$ and 101, both endpoints of $\rho$ , all six orderings of a contested workload, the two boundary-straddling traces, window rollover, revert atomicity, amount boundaries, constructor rejections, slot packing and the storage accesses a payment makes |
| ReferenceAnchors.t.sol | 10    | the same hand-computed traces replayed against the reference implementation                                                                                                                                                                                                                                                       |
| AdversarialToken.t.sol | 7     | tokens that re-enter, revert, or return false                                                                                                                                                                                                                                                                                     |
| Fuzz.t.sol             | 3     | apportionment against its defining inequality, and the debit-order transition                                                                                                                                                                                                                                                     |
| Invariant.t.sol        | 6     | I1, I3, I4, ghost agreement and the I2 probe, one campaign per configuration                                                                                                                                                                                                                                                      |
| Differential.t.sol     | 5     | both implementations on one call sequence, one campaign per configuration                                                                                                                                                                                                                                                         |
| Mutation.t.sol         | 1     | the four mutants against the named properties                                                                                                                                                                                                                                                                                     |
| RhoSweep.t.sol         | 1     | the $\rho$ sweep measurement                                                                                                                                                                                                                                                                                                      |
| DemoTrace.t.sol        | 1     | the six demo scenes, exported for the dashboard                                                                                                                                                                                                                                                                                   |

### 3.2 Invariant campaigns

Six configurations, each 64 runs of depth 128, so 8,192 handler calls per configuration. The handler pays and advances time; a refused payment is a normal outcome and does not end a run. Counts below are totals over all runs of the reference execution.

| $N$ | $B_G$      | $\rho$ | accepted | of those, spilled into surplus | refused | window rollovers |
|-----|------------|--------|----------|--------------------------------|---------|------------------|
| 2   | 1,000      | 0      | 1,800    | 1,800                          | 2,373   | 267              |
| 3   | 100        | 1/2    | 1,688    | 1,025                          | 2,485   | 264              |
| 3   | 100        | 1      | 1,766    | 0                              | 2,407   | 283              |
| 5   | 101        | 1/4    | 1,817    | 1,318                          | 2,356   | 276              |
| 10  | $2^{64}-1$ | 3/4    | 3,726    | 310                            | 447     | 316              |
| 50  | 1,000      | 1/3    | 2,810    | 1,812                          | 1,363   | 320              |

The refusal counts matter as much as the acceptances: a campaign in which nothing is ever refused never reaches the contested states the invariants are about.

### 3.3 Differential harness

The two implementations are deployed in the same block, so their windows coincide. Every handler call issues the same payment from the same caller to both, then compares the success flag and, on refusal, the raw return data. Both declare the same error signatures, so identical refusals produce identical selectors and arguments. After every call the harness compares the current window id, surplus consumption, the reserved total and cap, each delegate's spending, reservation and index, and the token balance each implementation has paid out. One caller in sixteen is an address that was never registered, and one amount in eight falls outside  $[1, B_G]$ , so the guard paths are exercised too.

| $N$ | $\rho$ | calls | accepted | refused | rollovers | unregistered callers | out-of-range amounts |
|-----|--------|-------|----------|---------|-----------|----------------------|----------------------|
| 2   | 0      | 8,321 | 1,656    | 2,517   | 297       | 554                  | 786                  |
| 3   | 1/2    | 8,322 | 1,584    | 2,590   | 298       | 537                  | 769                  |
| 3   | 1      | 8,321 | 1,399    | 2,774   | 272       | 568                  | 810                  |
| 5   | 1/4    | 8,321 | 1,484    | 2,689   | 269       | 552                  | 756                  |
| 10  | 3/4    | 8,321 | 2,679    | 1,494   | 304       | 555                  | 775                  |

A harness that always passes says nothing until it has been shown to fail. Breaking the reference implementation on purpose, by consuming the surplus before the reservation, is caught inside one campaign, with the reason reported as two refusals carrying different arguments. The README records how to repeat that check.

### 3.4 Mutation

Four copies of the deployed contract, each generated from the source with one behaviour changed:

| Mutant | Change                                                                                            |
|--------|---------------------------------------------------------------------------------------------------|
| M1     | the surplus is consumed before the delegate's own reservation                                     |
| M2     | the window tag comparison is dropped on the payment path, so stored values are used as they stand |
| M3     | a payment larger than the remaining surplus is filled partially instead of refused                |
| M4     | the reentrancy guard is removed                                                                   |

Each is run against the named properties, which were rewritten to return a verdict instead of asserting, so a failure is recorded rather than ending the run. The test asserts that all properties hold on the real contract and that no mutant survives all of them; the matrix is printed.

| Property                     | SpendPartition | M1            | M2            | M3            | M4            |
|------------------------------|----------------|---------------|---------------|---------------|---------------|
| apportionment at $\rho = 1$  | held           | held          | held          | <b>broken</b> | held          |
| debit order                  | held           | <b>broken</b> | held          | held          | held          |
| window reset                 | held           | held          | <b>broken</b> | held          | held          |
| ordering under a shared pool | held           | held          | held          | <b>broken</b> | held          |
| revert atomicity             | held           | held          | held          | <b>broken</b> | held          |
| one call, one payment        | held           | held          | held          | held          | <b>broken</b> |

Three of the four are caught by exactly one property each. Delete the debit-order test, the window-reset test or the reentrancy test and the corresponding bug passes the other fifty-two.

M4 is the interesting one. A token that calls back into the payment path on its own account is refused regardless of the guard, because it was never registered as a delegate; the attacker has to be a delegate that is itself a contract. Even then the aggregate bound is not what breaks. The contract commits its state before the transfer, so a nested payment reads the already-updated spending and is accounted like any other, and per-window  $\Sigma \text{spend} \leq B_G$  holds with the guard removed. What the guard buys is narrower: one call to the entry point performs exactly one payment. Without it the same call emits two. We wrote the property that way after the first version of the test failed to catch the mutant.

### 3.5 Tokens that misbehave

| Case                                                            | What is asserted                                                                                                         |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| the token re-enters pay during the transfer                     | the nested call reverts with the guard's error, whether the token bubbles it or swallows it                              |
| a delegate that is a contract re-enters under its own authority | the nested call fails, exactly one payment event is emitted, and the merchant balance equals the recorded spending       |
| a view is read during the callback                              | it returns the already-updated spending: there is no interval in which value is moving but the accounting has not landed |
| transfer reverts                                                | the payment reverts and both storage slots are byte-identical to before                                                  |
| transfer returns false                                          | SafeERC20 raises its failure error and no state moves                                                                    |

## 4. Measurements

Gas comes from receipts on a local anvil chain with every payment sent as its own transaction, so each pays the real cold-access costs. Deployment and payment figures are therefore what a user would pay, intrinsic cost included.

### 4.1 Payment and deployment against the delegate count

|                                                  | $N=2$   | $N=5$   | $N=10$  | $N=20$    | $N=50$    |
|--------------------------------------------------|---------|---------|---------|-----------|-----------|
| payment, reservation only                        | 47,123  | 47,123  | 47,123  | 47,123    | 47,123    |
| payment, surplus, first write of the contract    | 69,737  | 69,737  | 69,737  | 69,737    | 69,737    |
| payment, surplus, later write in the same window | 52,637  | 52,637  | 52,637  | 52,637    | 52,637    |
| deployment                                       | 800,128 | 875,002 | 999,780 | 1,249,450 | 1,998,732 |

Payment cost does not move with  $N$ , and it does not depend on which delegate pays or on how many payments came before. Deployment grows by 24,971.5 gas per registered delegate by least squares, which is one cold slot written per delegate plus its registration event. The work that depends on  $N$  is paid once, at setup.

### 4.2 The three surplus write regimes

Measured on contracts with a one-second window, with the chain clock advanced between payments so each lands in a window of its own: a surplus slot that has never been written costs 69,737, a later write in the same window costs 52,637, and the first write after a window boundary also costs 52,637. Only the first is expensive, because only it turns a zero slot non-zero. A payment sized exactly one unit past the payer's reservation, so that it straddles the boundary, lands in the same bracket as any other spilling payment.

### 4.3 The plain implementation as a cost baseline

|                    | $N=2$   | $N=5$   | $N=10$  | $N=20$    | $N=50$    |
|--------------------|---------|---------|---------|-----------|-----------|
| payment, packed    | 47,123  | 47,123  | 47,123  | 47,123    | 47,123    |
| payment, plain     | 81,935  | 85,113  | 93,972  | 118,842   | 193,452   |
| deployment, packed | 800,128 | 875,002 | 999,780 | 1,249,450 | 1,998,732 |
| deployment, plain  | 735,201 | 811,260 | 942,013 | 1,218,543 | 2,168,269 |

The plain version scans the delegate list on every payment, so its cost has a slope where the packed version has none. Deployment goes the other way at small  $N$ , where the plain version's shorter bytecode wins, and crosses over before fifty delegates.

### 4.4 Batching

A delegate that is a contract can issue  $k$  payments inside one transaction. The same  $k$  payments as separate transactions pay the intrinsic cost  $k$  times and touch every slot cold each time.  $N = 10$ :

| $\rho$ | $k$ | batched, one transaction | separate, $k$ transactions | per payment, batched | per payment, separate |
|--------|-----|--------------------------|----------------------------|----------------------|-----------------------|
| 0      | 1   | 56,529                   | 52,637                     | 56,529               | 52,637                |
| 0      | 4   | 86,499                   | 210,548                    | 21,625               | 52,637                |
| 0      | 16  | 206,379                  | 842,192                    | 12,899               | 52,637                |
| 1      | 1   | 51,015                   | 47,123                     | 51,015               | 47,123                |
| 1      | 4   | 78,843                   | 188,492                    | 19,711               | 47,123                |
| 1      | 16  | 190,155                  | 753,968                    | 11,885               | 47,123                |

At  $k = 1$  batching costs more, because of the extra call frame and its calldata. From  $k = 2$  the per-transaction costs that do not repeat inside a batch take over. One more payment inside a batch costs about 9,990 gas at  $\rho = 0$  and 9,276 at  $\rho = 1$ .

#### 4.5 What $\rho$ buys and what it costs

One fixed workload,  $N = 3$  and  $B_G = 100$  in a single window. Delegate A submits ten requests of 10 first, then B and C submit three each, so demand is 160 against a budget of 100. This is measurement: the test asserts only that the budget holds and that no request inside a delegate's own reservation is refused.

| $\rho$ | $r_i$ | A granted | B granted | C granted | total served |
|--------|-------|-----------|-----------|-----------|--------------|
| 0      | 0     | 100       | 0         | 0         | 100          |
| 0.2    | 7     | 80        | 10        | 10        | 100          |
| 0.4    | 14    | 70        | 10        | 10        | 90           |
| 0.5    | 17    | 60        | 20        | 20        | 100          |
| 0.7    | 24    | 50        | 20        | 20        | 90           |
| 0.8    | 27    | 40        | 30        | 30        | 100          |
| 1      | 34    | 30        | 30        | 30        | 90           |

The first mover's share falls from the whole budget to a third as  $\rho$  rises, and the later delegates go from nothing to a guaranteed share. Total served is not monotone: at  $\rho = 0.4, 0.7$  and 1 it drops to 90, because requests are atomic and a remainder smaller than one request cannot be used by anyone. That effect is a property of indivisible requests, not of the mechanism, and it is worth saying out loud rather than smoothing over.

#### 4.6 The six demo scenes

Produced by calling the contract, then reading the state back:

| Scene | Configuration               | Requests         | Outcome                                                       |
|-------|-----------------------------|------------------|---------------------------------------------------------------|
| 1     | caps of 80 each, intent 100 | A 80, B 80       | both pass, 160 leaves                                         |
| 2     | static 50/50 of 100         | A 80, then A 50  | 80 refused, 50 passes, half the budget idle                   |
| 3     | shared pool of 100          | A 80             | passes                                                        |
| 4     | shared pool of 100          | A 100, B 20      | A takes everything, B refused                                 |
| 5     | $\rho = 1/2$ of 100         | A 76, A 75, B 25 | 76 refused, 75 passes, B's 25 still served, total exactly 100 |
| 6     | the $\rho$ sweep above      |                  | the slider in the dashboard                                   |

results/dashboard.html is one self-contained file that renders all six from this data. It opens in a browser with no server and can be sent as a single attachment. The front end only displays: none of the rules are reimplemented in it.

### 5. Reproducing all of it

```
git clone https://github.com/liukefan821/spendpartition && cd spendpartition
forge install foundry-rs/forge-std@v1.16.2 OpenZeppelin/openzeppelin-contracts@v5.7.0
forge build && forge test # 53 passed, about 75 seconds
./run_gas_sweep.sh # starts anvil, broadcasts, prints the gas tables
python3 analysis/build_dashboard.py # writes results/dashboard.html
```

The gas figures are fixed by the bytecode and the EVM rules, both pinned, so a local run matches the stored reference column for column. The README gives the two diff commands that check this; the run recorded in results/container\_2026-09-30/ was reproduced on a different machine and a different Foundry version without a single digit changing.

### 6. What we checked in the literature and in other people's code

We read the two closest systems in source rather than in their documentation, because the scope of a counter is exactly where descriptions tend to be loose. SmartSessions keys its ERC-20 spending limit by session id, multiplexer, token and sender, so two sessions under one account keep independent counters. The experimental ERC-7579 SpendingLimitHook stores one limit and one spend counter per account and token, over a period hard-coded to one week, and only inspects calls whose selector is transfer; it has no delegate dimension at all. The MetaMask enforcers were checked against the official documentation in August.

Of the two off-chain systems, AIP states in §3.4 that "aggregate spend enforcement is the runtime's responsibility, not the token's", which is our gap described from the other side. CapLease keeps durable authorization state per authorization instance, off chain.

For the classical result we cite Kamoun and Kleinrock, IEEE Transactions on Communications COM-28(7), 1980. Its metadata is confirmed but the full text is behind a paywall and we have not read it, so nothing is quoted from it; the names of the three allocation policies are taken from Cidon et al., IEEE JSAC 13(7), 1995, which we did read. docs/citation\_check\_2026-09-23.md records each of these checks with its date.

## 7. What is not done

---

The checklist in the frozen storage-layout document is closed. What remains is work that starts once the scope is approved: splitting the four workstreams across the team, the written report, the ten-minute video, and optionally a public testnet deployment. Nothing in the repository depends on approval to keep running, and nothing built so far assumes a particular verdict on scope.